

Automata and Formal Languages

Dr. Sankhadeep Chatterjee

Context Free Grammars

- A *context-free grammar* is a notation for describing languages.
- It is more powerful than finite automata or RE's, but still cannot define all possible languages.
- Useful for nested structures, e.g., parentheses in programming languages.
- Basic idea is to use “variables” to stand for sets of strings (i.e., languages).
- These variables are defined recursively, in terms of one another.
- Recursive rules (“productions”) involve only concatenation.
- Alternative rules for a variable allow union.

Grammars

- *Terminals* = symbols of the alphabet of the language being defined.
- *Variables* = *nonterminals* = a finite set of other symbols, each of which represents a language.
- *Start symbol* = the variable whose language is the one being defined.
- A *production* has the form variable $\rightarrow$ string of variables and terminals.
- Convention:
 - A, B, C,... are variables.
 - a, b, c,... are terminals.
 - ..., X, Y, Z are either terminals or variables.
 - ..., w, x, y, z are strings of terminals only.
 - α , β , γ ,... are strings of terminals and/or variables.

Example: Formal CFG

- Here is a formal CFG for $\{ 0^n 1^n \mid n \geq 1 \}$.
- Terminals = $\{0, 1\}$.
- Variables = $\{S\}$.
- Start symbol = S .
- Productions =
 - $S \rightarrow 01$
 - $S \rightarrow 0S1$

Derivations – Intuition

- We *derive* strings in the language of a CFG by starting with the start symbol, and repeatedly replacing some variable A by the right side of one of its productions.
 - That is, the “productions for A ” are those that have A on the left side of the $\rightarrow$.
- We say $\alpha A \beta \Rightarrow \alpha \gamma \beta$ if $A \rightarrow \gamma$ is a production.
- Example: $S \rightarrow 01$; $S \rightarrow 0S1$.
- $S \Rightarrow 0S1 \Rightarrow 00S11 \Rightarrow 000111$.



Derivations

- $\Rightarrow^*$ means “zero or more derivation steps.”
- Basis: $\alpha \Rightarrow^* \alpha$ for any string α .
- Induction: if $\alpha \Rightarrow^* \beta$ and $\beta \Rightarrow \gamma$, then $\alpha \Rightarrow^* \gamma$.
 - $S \rightarrow 01; S \rightarrow 0S1$.
 - $S \Rightarrow 0S1 \Rightarrow 00S11 \Rightarrow 000111$.
 - So $S \Rightarrow^* S; S \Rightarrow^* 0S1; S \Rightarrow^* 00S11; S \Rightarrow^* 000111$.
- Any string of variables and/or terminals derived from the start symbol is called a *sentential form*.
- Formally, α is a sentential form iff $S \Rightarrow^* \alpha$.

Language of CFG

- If G is a CFG, then $L(G)$, the *language of G* , is $\{w \mid S \Rightarrow^* w\}$.
 - Note: w must be a terminal string, S is the start symbol.
- Example: G has productions $S \rightarrow \epsilon$ and $S \rightarrow 0S1$.
- $L(G) = \{0^n 1^n \mid n \geq 0\}$.
- A language that is defined by some CFG is called a *context-free language*.
- There are CFL's that are not regular languages, such as the example just given.
- But not all languages are CFL's.

Leftmost and Rightmost Derivations

- Derivations allow us to replace any of the variables in a string.
- Leads to many different derivations of the same string.
- By forcing the leftmost variable (or alternatively, the rightmost variable) to be replaced, we avoid these “distinctions without a difference.”
- Say $wA\alpha \Rightarrow_{lm} w\beta\alpha$ if w is a string of terminals only and $A \rightarrow \beta$ is a production.
- Also, $\alpha \Rightarrow_{lm}^* \beta$ if α becomes β by a sequence of 0 or more $\Rightarrow_{lm}$ steps.

Example: Leftmost Derivations

- Balanced-parentheses grammar:

$$S \rightarrow SS \mid (S) \mid ()$$

- $S \Rightarrow_{lm} SS \Rightarrow_{lm} (S)S \Rightarrow_{lm} (())S \Rightarrow_{lm} (())()$

- Thus, $S \Rightarrow_{lm}^* (())()$

- $S \Rightarrow SS \Rightarrow S() \Rightarrow (S)() \Rightarrow (())()$ is a derivation, but not a leftmost derivation.

Rightmost Derivations

- Say $\alpha Aw \Rightarrow_{rm} \alpha \beta w$ if w is a string of terminals only and $A \rightarrow \beta$ is a production.
- Also, $\alpha \Rightarrow_{rm}^* \beta$ if α becomes β by a sequence of 0 or more $\Rightarrow_{rm}$ steps.
- Balanced-parentheses grammar:

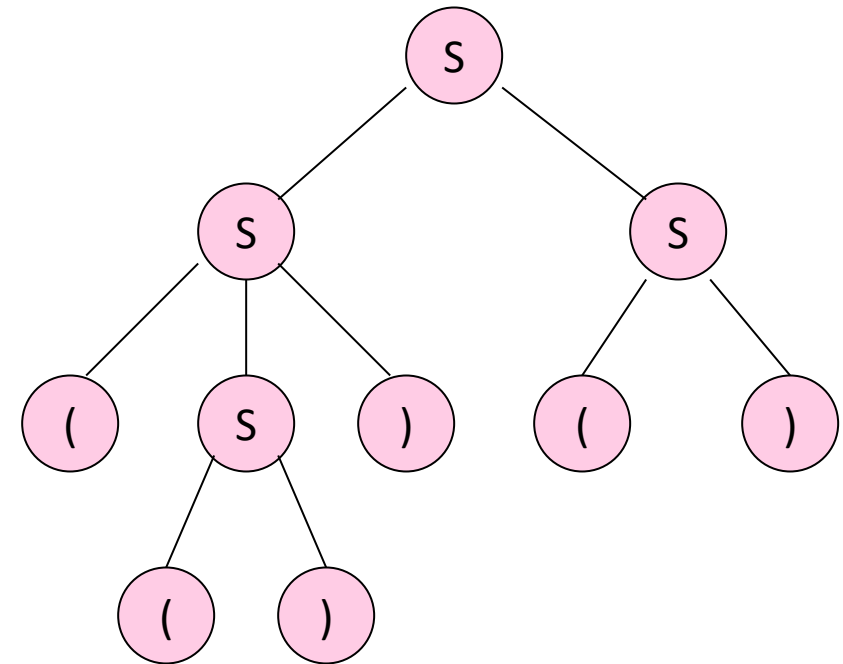
$$S \rightarrow SS \mid (S) \mid ()$$

- $S \Rightarrow_{rm} SS \Rightarrow_{rm} S() \Rightarrow_{rm} (S)() \Rightarrow_{rm} (())(())$
- Thus, $S \Rightarrow_{rm}^* (())(())$
- $S \Rightarrow SS \Rightarrow SSS \Rightarrow S()S \Rightarrow ()()S \Rightarrow ()()()$ is neither a rightmost nor a leftmost derivation.

Parse Trees

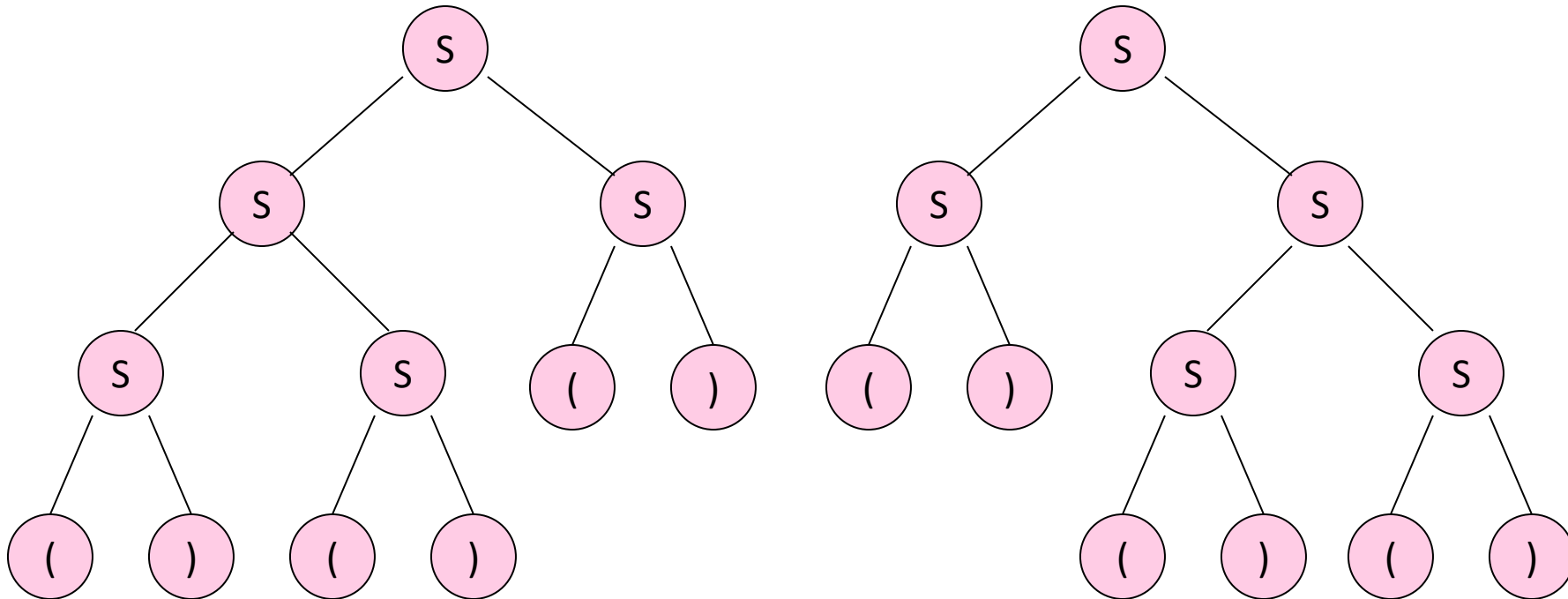
- *Parse trees* are trees labeled by symbols of a particular CFG.
- Leaves: labeled by a terminal or ϵ .
- Interior nodes: labeled by a variable.
 - Children are labeled by the right side of a production for the parent.
- Root: must be labeled by the start symbol.
- The concatenation of the labels of the leaves in left-to-right order
 - That is, in the order of a preorder traversal.is called the *yield* of the parse tree.
- Example: yield is $((\))(\))$
- For every parse tree, there is a unique leftmost, and a unique rightmost derivation.

$S \rightarrow SS \mid (S) \mid ()$



Ambiguous Grammars

- A CFG is *ambiguous* if there is a string in the language that is the yield of two or more parse trees.
- Example: $S \rightarrow SS \mid (S) \mid ()$
- Two parse trees for $()()()$



Ambiguity is a Property of Grammars, not Languages

- For the balanced-parentheses language, here is another CFG, which is unambiguous.

$B \rightarrow (RB \mid \epsilon$

$R \rightarrow) \mid (RR$

B, the start symbol,
derives balanced strings.

R generates strings that
have one more right paren
than left.

Variables That Derive Nothing

- Consider: $S \rightarrow AB$, $A \rightarrow aA \mid a$, $B \rightarrow AB$
- Although A derives all strings of a 's, B derives no terminal strings.
- Thus, S derives nothing, and the language is empty.

Testing Whether a Variable Derives Some Terminal String

- Basis: If there is a production $A \rightarrow w$, where w has no variables, then A derives a terminal string.
- Induction: If there is a production $A \rightarrow \alpha$, where α consists only of terminals and variables known to derive a terminal string, then A derives a terminal string.
- Eventually, we can find no more variables.
- An easy induction on the order in which variables are discovered shows that each one truly derives a terminal string.
- Conversely, any variable that derives a terminal string will be discovered by this algorithm.

Algorithm to Eliminate Variables That Derive Nothing

1. Discover all variables that derive terminal strings.
2. For all other variables, remove all productions in which they appear either on the left or the right.

3. Example:

$S \rightarrow AB \mid C, A \rightarrow aA \mid a, B \rightarrow bB, C \rightarrow c$

- Basis: A and C are identified because of $A \rightarrow a$ and $C \rightarrow c$.
- Induction: S is identified because of $S \rightarrow C$.
- Nothing else can be identified.
- Result: $S \rightarrow C, A \rightarrow aA \mid a, C \rightarrow c$

Unreachable Symbols

- Another way a terminal or variable deserves to be eliminated is if it cannot appear in any derivation from the start symbol.
- Basis: We can reach S (the start symbol).
- Induction: if we can reach A , and there is a production $A \rightarrow \alpha$, then we can reach all symbols of α .
- Algorithm: Remove from the grammar all symbols not discovered reachable from S and all productions that involve these symbols.

Eliminating Useless Symbols

- A symbol is *useful* if it appears in some derivation of some terminal string from the start symbol. Otherwise, it is *useless*.
- Eliminate all useless symbols by:
 1. Eliminate symbols that derive no terminal string.
 2. Eliminate unreachable symbols.

$S \rightarrow AB, A \rightarrow C, C \rightarrow c, B \rightarrow bB$

- If we eliminated unreachable symbols first, we would find everything is reachable.
- A, C, and c would never get eliminated.

Epsilon Productions

- We can almost avoid using productions of the form $A \rightarrow \epsilon$ (called *ϵ -productions*).
 - The problem is that ϵ cannot be in the language of any grammar that has no ϵ -productions.
- Theorem: If L is a CFL, then $L - \{\epsilon\}$ has a CFG with no ϵ -productions.
- To eliminate ϵ -productions, we first need to discover the *nullable variables* = variables A such that $A \Rightarrow^* \epsilon$.

Example: Nullable Symbols

- Basis: If there is a production $A \rightarrow \epsilon$, then A is nullable.
- Induction: If there is a production $A \rightarrow \alpha$, and all symbols of α are nullable, then A is nullable.

$S \rightarrow AB, A \rightarrow aA \mid \epsilon, B \rightarrow bB \mid A$

- Basis: A is nullable because of $A \rightarrow \epsilon$.
- Induction: B is nullable because of $B \rightarrow A$.
- Then, S is nullable because of $S \rightarrow AB$.

Eliminating ϵ -Productions

- Key idea: turn each production $A \rightarrow X_1 \dots X_n$ into a family of productions.
- Except, if all X 's are nullable, do not make a production with ϵ as the right side.

$S \rightarrow ABC, A \rightarrow aA \mid \epsilon, B \rightarrow bB \mid \epsilon, C \rightarrow \epsilon$

- A, B, C, and S are all nullable.
- New grammar:

$S \rightarrow \cancel{ABC} \mid AB \mid \cancel{AC} \mid \cancel{BC} \mid A \mid B \mid \cancel{C}$

$A \rightarrow aA \mid a$

$B \rightarrow bB \mid b$

- Note: C is now useless. Eliminate its productions.

Unit Productions

- A *unit production* is one whose right side consists of exactly one variable.
- These productions can be eliminated.
- Key idea:
 - If $A \Rightarrow^* B$ by a series of unit productions, and $B \rightarrow \alpha$ is a non-unit-production, then add production $A \rightarrow \alpha$.
 - Then, drop all unit productions.
- Find all pairs (A, B) such that $A \Rightarrow^* B$ by a sequence of unit productions only.
- Basis: Surely (A, A) .
- Induction: If we have found (A, B) , and $B \rightarrow C$ is a unit production, then add (A, C) .

Cleaning Up a Grammar

- Theorem: if L is a CFL, then there is a CFG for $L - \{\epsilon\}$ that has:
 1. No useless symbols.
 2. No ϵ -productions.
 3. No unit productions.
- I.e., every right side is either a single terminal or has length ≥ 2 .
- Perform the following steps in order:
 1. Eliminate ϵ -productions.
 2. Eliminate unit productions.
 3. Eliminate variables that derive no terminal string.
 4. Eliminate variables not reached from the start symbol.

Chomsky Normal Form

- A CFG is said to be in *Chomsky Normal Form* if every production is of one of these two forms:
 1. $A \rightarrow BC$ (right side is two variables).
 2. $A \rightarrow a$ (right side is a single terminal).
- Theorem: If L is a CFL, then $L - \{\epsilon\}$ has a CFG in CNF.

Example:

- Consider production $A \rightarrow BcDe$.
- We need variables A_c and A_e . with productions $A_c \rightarrow c$ and $A_e \rightarrow e$.
 - Note: you create at most one variable for each terminal, and use it everywhere it is needed.
- Replace $A \rightarrow BcDe$ by $A \rightarrow BA_cDA_e$.

Example – Continued

- Recall $A \rightarrow BCDE$ is replaced by $A \rightarrow BF$, $F \rightarrow CG$, and $G \rightarrow DE$.
- In the new grammar, $A \Rightarrow BF \Rightarrow BCG \Rightarrow BCDE$.
- More importantly: Once we choose to replace A by BF , we must continue to BCG and $BCDE$.
 - Because F and G have only one production.